BetterAuth Documentation

Pre-requisites:

- A valid Amazon AWS account with access to AWS Cognito
- Windows 10 or later or Windows Server 2016 or later
- An Intel Pentium 4 processor or later that is SSE3 capable

BetterAuth is implemented and functions through the collaboration of many different AWS services, config files, code, federated identity providers and their respective client IDs. AWS services involved with this implementation namely include: AWS Cognito and AWS Amplify. AWS Cognito provides the data for the Login page, and creates the page itself. This begins with the "User pool" in Cognito, where an App Client is created for the webpage, where details about the web page are provided. Callback and sign-out URLs also need to be configured for the App Client, which are the URLs used for the web app, so considering the domain name for the Clockwork MVP Web App is "lab.crusaders.clockwork.vip", with "/login" attached for the login URL, and "/oauth2/idresponse" attached for the login response URL, the following are used for the URLs:

Allowed callback URLs

http://localhost:3000/

https://lab.crusaders.clockwork.vip

https://lab.crusaders.clockwork.vip/login

https://lab.crusaders.clockwork.vip/oauth2/idpresponse

Allowed sign-out URLs

http://localhost:3000/

https://lab.crusaders.clockwork.vip

The login will only work however if federated identity providers are also created and added to the respective location in AWS Cognito, because they provide the details to connect the webpage to a login provider. The details cannot be added however without creating developer accounts and clients for the login providers first. This is completed through going to the login provider's developer webpage and completing the account creation process, and then going to the security profiles to

create a client. Although, a web app can be deployed by simply creating a client for every federated identity provider, each client needs to have its URLs configured similarly to the URLs for the App Client, to ensure logging in with function successfully. However, certain additions are made according to the login URL seen for the login page created within AWS Cognito, accessible via the ”View login page” button under “Login pages” in “App clients”. These are seen for the ClockWork MVP Web App login page with:

Google:

URI 1:

https://userpool-ezsa3.auth.ap-southeast-2.amazoncognito.com/oauth2/idpresponse

URI 2:

https://ap-southeast-2do0zvzlo7.auth.ap-southeast-2.amazoncognito.com/oauth2/idpresponse

URI 3:

https://lab.crusaders.clockwork.vip/

URI 4:

https://lab.crusaders.clockwork.vip/oauth2/idpresponse

LoginWithAmazon:

Allowed Origins
- https://lab.crusaders.clockwork.vip

Allowed Return URLs
- https://lab.crusaders.clockwork.vip/oauth2/idpresponse

- https://ap-southeast-2do0zvzlo7.auth.ap-southeast-2.amazoncognito.com/oauth2/idpresponse

The details for the client are then provided for the respective federated identity provider. Scopes for the details to be passed to the webpage also need to be outlined to create a federated identity provider in AWS, these scopes need to be carefully chosen, as certain selection of scopes can affect the webpage's functionality, or even cause it to stop working entirely. A lot of the details provided for AWS Cognito need to be updated in the webpage's configuration files, as mismatching between AWS and the files may occur. This is safe, despite the warning at the top of aws-exports.js that says specifically not to do so, if the updated details are correct and according to how AWS Cognito is setup. The parts of config files that needed to be updated for the ClockWork MVP Web App included:

aws-exports.js:

```
  "aws_user_pools_web_client_id": "un0t3utbt8p4a77jfang6fe3p",
  "oauth": {
    "domain": "ap-southeast-2do0zvzlo7.auth.ap-southeast-2.amazoncognito.com",
    "scope": [
      "phone",
      "email",
      "profile"
    ],
    "redirectSignIn": "https://lab.crusaders.clockwork.vip/oauth2/idpresponse",
    "redirectSignOut": "https://lab.crusaders.clockwork.vip/",
    "responseType": "code"
  },
  "federationTarget": "COGNITO_USER_POOLS",
```

```
  "aws_cognito_username_attributes": [
    "EMAIL"
  ],
  "aws_cognito_social_providers": [
    "GOOGLE",
    "AMAZON"
  ],
  "aws_cognito_signup_attributes": [
    "EMAIL"
  ],
```


amplifyconfiguration.json:


```
 "aws_user_pools_web_client_id": "un0t3utbt8p4a77jfang6fe3p",
 "oauth": {
  "domain": "ap-southeast-2do0zvzlo7.auth.ap-southeast-2.amazoncognito.com",
  "scope": [
   "phone",
   "email",
   "profile"
  ],
  "redirectSignIn": "https://lab.crusaders.clockwork.vip/oauth2/idpresponse",
  "redirectSignOut": "https://lab.crusaders.clockwork.vip/",
```

```
  "responseType": "code"
 },
 "federationTarget": "COGNITO_USER_POOLS",
 "aws_cognito_username_attributes": [
  "EMAIL"
 ],
 "aws_cognito_social_providers": [
  "GOOGLE",
  "AMAZON"
 ],
 "aws_cognito_signup_attributes": [
  "EMAIL"
 ],
```

backend-config.json:

```
    "signupAttributes": [
     "EMAIL"
    ],
    "socialProviders": [
     "GOOGLE",
     "AMAZON"
    ],
```

```
    "usernameAttributes": [
      "EMAIL"
    ],
    "verificationMechanisms": [
      "EMAIL"
    ]
```

The code for the login page(s) needs to be updated to support all this configuration and setup, with the best, up-to-date option for implementation being using the import “{ signInWithRedirect, signOut } from "aws-amplify/auth"”

This is implemented by adding a function that calls the signInWithRedirect method in an await for loading time, inside a try/catch, along with details tailored to your web page, and attaching it to a button. This is seen in the ClockWork MVP Web App Login.tsx with:

```
const handleFederatedLogin = async (provider: Provider) => {
  try {
    await signInWithRedirect({
      provider: provider === Provider.GOOGLE ? "Google" : { custom: "LoginWithAmazon" },
    });
  } catch (error) {
    console.error("Federated sign-in error:", error);
  }
```

AWS Amplify also needs to be setup meticulously to host a web app. This is seen with filling out its details to ensure the webpage’s low downtime.